

Detecting AI-Generated Product Images on the Etsy Marketplace: A Comparative Study of Hybrid Feature Extraction and End-to-End Fine-Tuning

Megha Kanojia
School of Computing
Dublin City University
Dublin, Ireland
megha.kanojia2@mail.dcu.ie

Shubin Li
School of Computing
Dublin City University
Dublin, Ireland
shubin.li2@mail.dcu.ie

Abstract—Generative models such as Midjourney, DALL-E, and Stable Diffusion now allow sellers on online marketplaces to pass off AI-generated images as genuine product photographs. We study the binary classification problem of telling the two apart on the Etsy marketplace, evaluated by F1 score on a hidden test set. Our work spans three phases. Phase 1 uses a frozen ImageNet-pretrained CNN as a feature extractor feeding into classical classifiers (SVM, Random Forest, Gradient Boosting). Phase 2 fine-tunes a modern backbone end-to-end: our EfficientNetV2-S reaches a validation F1 of 0.9076, 9.4 points above the best Phase 1 result (ResNet50 features + tuned SVM, 0.8134). Phase 3 builds a three-model ensemble of EfficientNetV2-S with an FFT frequency branch, ConvNeXt-Small, and ConvNeXt-Base, trained with Focal Loss, Mixup, Stochastic Weight Averaging, and 5-view Test-Time Augmentation, pushing validation F1 to 0.9317. Alongside the main results, we examine three techniques commonly recommended for small-data image tasks: aggressive data augmentation, test-time augmentation (TTA), and threshold tuning, and report their negative or marginal outcomes on the single model: stronger augmentation dropped F1 by 1.3 points, TTA by 0.23 points, and threshold tuning gained only 0.09 points, which is within statistical noise on a 960-image validation set. The single-model findings and the ensemble outcome together suggest that on medium-sized forensic datasets with subtle texture signals, end-to-end fine-tuning of a compact backbone is a strong default, while inference-time tricks help only when there is model-level diversity to build on.

Index Terms—AI-generated image detection, transfer learning, EfficientNetV2, ensemble learning, ConvNeXt, focal loss

I. INTRODUCTION

Online marketplaces such as Etsy bring independent sellers together with buyers across the world, and the whole system runs on trust. Recent generative image models have created a new kind of fraud: sellers can post AI-generated images as if they were real product photos, listing items they have never actually made. Automated detection of such images is therefore essential for marketplace integrity. We address this task as a supervised binary image classification problem: given a listing image, predict whether it is AI-generated (label = 1) or an authentic photograph (label = 0). The performance metric is the F1 score on a hidden test set whose labels are not visible to participants.

The task comes with three difficulties. First, the dataset is not large, about 4800 training images; so training a deep model from scratch is not realistic, and transfer learning becomes the natural choice. Second, the signal to detect is subtle. Unlike ordinary object recognition, where the target has a clear shape, AI-generated artifacts show up as slight texture irregularities, off color statistics, and patterns in the frequency domain. Third, the AI images in the dataset likely come from several different generators, each leaving its own kind of trace.

The contributions of this work are threefold:

- We conduct a three-phase study: Phase 1 explores hybrid feature extraction (frozen backbone + classical ML head), and Phase 2 explores end-to-end fine-tuning of a modern backbone. We demonstrate that Phase 2 substantially outperforms Phase 1 on this task.
- We show that EfficientNetV2-S fine-tuned with a minimal augmentation pipeline achieves a validation F1 of 0.9076, exceeding the best Phase 1 hybrid model by 9.4 percentage points.
- We conduct rigorous ablation studies on three commonly-prescribed enhancements, aggressive augmentation, test-time augmentation (TTA), and threshold tuning and report their failure or marginal effect on this task. We discuss the implications and offer principled explanations grounded in the nature of the data.
- We introduce a Phase 3 ensemble approach combining architecturally diverse backbones (EfficientNetV2-S, ConvNeXt-Small, ConvNeXt-Base) with a frequency-domain FFT branch, Focal Loss, Mixup, SWA, and 5-view TTA, achieving a validation F1 of 0.9317.

II. RELATED WORK

A. EfficientNetV2

The EfficientNetV2 family [1] extends the original EfficientNet [2] by replacing the early depthwise convolutions with fused MB-Conv blocks for better GPU throughput, and by adopting progressive learning during training. EfficientNetV2-S has roughly 22 million parameters and a recommended input

resolution of 384×384 , and delivers competitive ImageNet accuracy without the compute or memory cost of much larger models.

B. Test-time augmentation and threshold tuning

Test-time augmentation (TTA) [3] averages the model's predictions across several augmented copies of a test image to reduce prediction variance. Threshold tuning searches for the decision threshold on a held-out set that best optimizes a chosen metric such as F1, and is often used when the positive and negative classes are imbalanced. We evaluate both techniques in Section V.

III. DATASET AND PREPROCESSING

A. Dataset

The dataset provides 4800 labeled training images and a hidden test set of 2058 images. Each training row lists an image ID and its ground-truth label (0 = authentic, 1 = AI-generated). Images are stored in a single archive with filenames matching the image IDs.

B. Train/Validation Split

We split the training set 80/20 using a stratified scheme with seed 42, yielding about 3840 images for training and 960 for validation. The resulting validation split contains 497 authentic images (51.8%) and 463 AI-generated images (48.2%), giving a nearly balanced binary task.

IV. METHODOLOGY

A. Phase 1: Hybrid Feature Extraction Approaches

a) *Pixel baseline*: Images are resized to 32×32 and flattened into a 3072-dimensional vector of normalised RGB pixel values (divided by 255). A Random Forest (100 estimators) trained on these raw features serves as the lower-bound reference, achieving a validation F1 of 0.6229.

b) *Hand-crafted features + classical ML*: Images are resized to 64×64 and represented by a concatenation of three feature groups: YCrCb colour channels (flattened, 12288-d), Sobel edge magnitude (4096-d), and per-channel colour histograms (256 bins $\times$ 3 channels = 768-d), yielding a 17,152-dimensional vector. Three classifiers are evaluated on this representation: Histogram Gradient Boosting (max_iter=200, lr=0.1, l2=1.5) achieving F1 0.6438; SVM with RBF kernel (C=1.0, PCA to 100 components) achieving F1 0.6411; and Random Forest (200 estimators, balanced class weight) achieving F1 0.6525.

c) *ResNet50 features + classical ML*: A frozen ImageNet-pretrained ResNet50 (include_top=False, global average pooling) extracts a 2048-dimensional embedding per image at 224×224 resolution. The same three classifiers are applied to these deep features: Random Forest achieves F1 0.7540; Gradient Boosting achieves F1 0.7809; SVM (RBF, C=1.0, balanced) achieves F1 0.8134, the strongest Phase 1 result.

The consistent pattern across Phase 1 is that deep CNN features (ResNet50, 2048-d) substantially outperform both

pixel and hand-crafted features, and SVM with an RBF kernel outperforms tree-based classifiers on those deep embeddings. The best Phase 1 result, ResNet50 + SVM, F1 = 0.8134, establishes the baseline that Phase 2 and Phase 3 aim to exceed.

B. Phase 2: End-to-End Fine-Tuning of EfficientNetV2-S

- **Backbone selection.** We use `tf_efficientnetv2_s` from the `timm` library [4]: ImageNet-pretrained, about 22M parameters, and a 384×384 input size. Compared with ResNet50 (about 25M parameters, 224×224 input, from 2015), EfficientNetV2-S has a similar parameter budget but reaches higher ImageNet accuracy and trains more efficiently. The larger 384×384 input also preserves finer spatial detail, which should help when the target signal is a subtle texture artifact.
- **Architecture.** We build the model through `timm`'s `create_model` API with `num_classes = 1`, so the original 1000-way classifier is swapped for a single-logit head. The model is trained as a binary classifier with `BCEWithLogitsLoss`, which combines a sigmoid and binary cross-entropy in one numerically stable step.
- **Optimization.** We train with Adam at a learning rate of 1×10^{-4} , a batch size of 24 (chosen to fit inside the 12 GB memory of an RTX 4070 Ti at 384×384 input), for 10 epochs. We do not use a learning rate schedule, weight decay, mixed precision, or a two-stage freeze/unfreeze setup. The pipeline is kept deliberately simple, so we have a clean reference point before adding any extra tricks.
- **Augmentation.** Training uses random horizontal flipping only. At validation and test time we resize and normalize the inputs without any augmentation.
- **Model selection.** At the end of each epoch we compute validation F1 at the default threshold of 0.5, and we save the checkpoint with the highest value seen so far. The resulting best-epoch weights are used for every later analysis and for the final test submission.

C. Inference-Time Enhancements (Investigated)

- **Test-Time Augmentation (TTA).** At inference time, each image is forwarded through the model twice, once as the original image and once horizontally flipped, and the resulting sigmoid probabilities are averaged before thresholding.
- **Threshold tuning (no TTA).** We sweep the decision threshold over [0.30, 0.70] in increments of 0.01 on the validation set, and select the threshold yielding the highest F1.
- **Combined TTA + threshold tuning.** Perform the same threshold search on the TTA-averaged probabilities.

D. Phase 3: Multi-Model Ensemble with Frequency-Domain Features

- **Architecture: three models.**
 - **Model A:** EfficientNetV2-S backbone + FFT branch. The FFT branch converts each image to grayscale,

applies 2D FFT, takes log-magnitude, resizes to 64×64, and passes through a 2-layer MLP (4096→256→128). The FFT and backbone outputs are concatenated before the classification head.

- Model B: ConvNeXt-Small [5], pretrained on ImageNet, num_classes=0 global average pooled, with a 2-layer head (dropout→Linear→ReLU→dropout→Linear).
- Model C: ConvNeXt-Base [5], same architecture as B but 89M parameters vs 50M.

• **Training configuration (applies to all three models):**

- Loss: Focal Loss ($\gamma=2.0$) with positive class weighting (neg/pos ratio ≈ 1.07) [6]
- Optimiser: AdamW with two-speed LR: backbone 2×10^{-5} , head/FFT branch 1×10^{-4} , weight decay 1×10^{-4}
- Scheduler: OneCycleLR (pct_start=0.1) for epochs 1-13, switching to SWALR (1×10^{-5}) from epoch 14 onward
- Epochs: 16 total; SWA active from epoch 14. [7]
- Mixed precision (torch.cuda.amp) and gradient clipping (max norm 1.0)
- Mixup augmentation applied with probability 0.5 per batch ($\alpha=0.4$) [8]
- Model C batch size: 8 (T4 memory constraint); Models A and B: 16

- **Augmentation pipeline (new, differs from Phase 2):** Resize to 416→RandomCrop to 384, RandomHorizontalFlip, RandomVerticalFlip, ColorJitter (brightness/contrast 0.3, saturation 0.2, hue 0.05), JPEG compression simulation (quality 40–85, p=0.5), RandomGrayscale (p=0.05), RandomErasing (p=0.2, scale 0.02–0.1).

- **Ensemble strategy:** Three strategies evaluated on validation with 5-view TTA: equal average, F1-proportional weighted, rank-squared weighted. Best selected automatically.

- **5-view TTA views:** Original, horizontal flip, center crop from 416→384, $\pm 7^\circ$ rotation, mild colour jitter.

V. EXPERIMENTS AND RESULTS

A. Experimental Setup

All reported F1 scores are computed on the held-out validation split of 960 images, treating AI-generated images as the positive class. To ensure reproducibility, random seeds are fixed at 42 across Python, NumPy, and PyTorch (both CPU and CUDA).

Phase 1 setup. All Phase 1 experiments are conducted on Google Colab (T4 GPU, 15 GB RAM) using Python 3.10, scikit-learn 1.3, TensorFlow 2.15 (for ResNet50 feature extraction), and OpenCV 4.8.

Phase 2 setup. All Phase 2 experiments are conducted on a single NVIDIA RTX 4070 Ti GPU with 12 GB of memory, using PyTorch 2.5.1 with CUDA 12.1 and the timm library [4].

Phase 3 setup. All Phase 3 experiments are conducted on Google Colab (T4 GPU, 15 GB RAM) using PyTorch 2.x with CUDA, the timm library, and torch.optim.swa_utils for Stochastic Weight Averaging.

B. Main Results

Phase 1: Hybrid Feature Extraction Approaches

Hand-crafted: YCrCb + Sobel + colour histograms (17,152-d)

TABLE I
PERFORMANCE COMPARISON OF BASELINE MODELS IN PHASE 1

Feature Representation	Classifier	Val F1
Raw pixels (32×32 , 3072-d)	Random Forest (100 est.)	0.6229
Hand-crafted	Gradient Boosting	0.6438
Hand-crafted	SVM (RBF, PCA-100)	0.6411
Hand-crafted	Random Forest (200 est.)	0.6525
ResNet50 features (2048-d, frozen)	Random Forest (200 est.)	0.7540
ResNet50 features (2048-d, frozen)	Gradient Boosting	0.7809
ResNet50 features (2048-d, frozen)	SVM (RBF, C=1.0)	0.8134

The jump from hand-crafted (best: 0.6525) to ResNet50 features (best: 0.8134) is +0.161 F1, confirming that frozen deep CNN embeddings are vastly more discriminative than manually engineered descriptors for this task. Within each feature set, SVM with an RBF kernel consistently outperforms tree-based methods.

Phase 2: End-to-End Fine-Tuning of EfficientNetV2-S

End-to-end fine-tuning of EfficientNetV2-S reaches a validation F1 of 0.9076, 9.4 percentage points above the strongest Phase 1 hybrid configuration. This gap comes from allowing the backbone itself to adapt to AI-artifact features, something a frozen backbone cannot do no matter which classifier is placed on top. Setup and ablations are described in Sections IV-B and V-C, V-D.

Phase 3: Ensemble combining EfficientNetV2-S with an FFT frequency branch, ConvNeXt-Small, and ConvNeXt-Base

- ConvNeXt-Base is the strongest single model at 0.9356, exceeding even the ensemble. It suggests the base model’s larger receptive fields are capturing the texture artifacts very effectively on their own.
- SWA did not improve Model A (SWA F1 = 0.8875 vs regular best 0.8930) but it was active when Model B achieved its best epoch (epoch 16, marked [SWA], F1 = 0.9201). SWA effectiveness appears model-dependent.
- The ensemble (0.9317) is actually lower than ConvNeXt-Base alone (0.9356). The weaker models (especially Model A at 0.8930) pulled the ensemble average down. This directly contradicts the common intuition that “more models always help” and supports the conclusion that ensemble diversity quality matters more than quantity.
- Focal Loss caused high thresholds, Model A best threshold was 0.63, Model B 0.63, Model C 0.44. This is a known calibration effect of Focal Loss.

TABLE II
PERFORMANCE COMPARISON OF MODELS IN PHASE 3

Model	Epochs	Val F1	Best epoch
Model A (EffNetV2-S + FFT)	16	0.8930	8
Model B (ConvNeXt-Small)	16	0.9201	16 (SWA)
Model C (ConvNeXt-Base)	16	0.9356	7
Equal ensemble (3-model + TTA)	-	0.9310	-
Weighted ensemble	-	0.9310	-
Rank-squared ensemble	-	0.9317	-

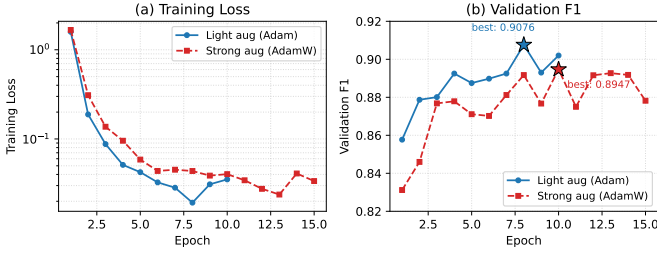


Fig. 1. Training dynamics of the two Phase 2 runs. (a) Training loss on a log scale. (b) Validation F1 per epoch; stars mark the best checkpoint of each run.

C. Ablation: Data Augmentation Strength

A widely cited heuristic in deep learning is that small datasets benefit from strong data augmentation. We tested this hypothesis with an aggressive augmentation pipeline.

Strong augmentation configuration. The pipeline consists of Resize to 416, RandomCrop to 384, RandomHorizontalFlip, ColorJitter (brightness, contrast, saturation all set to 0.2), Normalize, and RandomErasing with probability 0.25 and scale range (0.02, 0.15). We additionally switched the optimizer from Adam to AdamW with weight decay 1×10^{-4} , and increased training to 15 epochs to allow the model to converge under the more challenging augmentation regime.

As shown in Table III, the stronger augmentation pipeline produced a 1.3 percentage point decrease in validation F1, even with five additional training epochs. Figure 1 shows the training dynamics of both runs in more detail: although the strong-augmentation run eventually drives training loss to a comparable level, its validation F1 plateaus roughly one percentage point below the light-augmentation run and never catches up, even given 50% more epochs.

We propose two complementary explanations:

- **Local artifact preservation.** AI-generated artifacts tend to manifest as subtle, spatially-localized texture inconsistencies. RandomCrop and RandomErasing remove or perturb image regions that may contain the precise artifacts the model needs to learn from. ColorJitter, similarly, distorts color statistics that are themselves potentially discriminative between authentic and synthetic images.
- **Sufficiency of dataset size.** With approximately 3840 training images and a backbone of 22M parameters, the dataset is not so small that the model is starved

TABLE III
EFFECT OF AUGMENTATION STRENGTH ON VALIDATION F1.

Config	Augmentation	Optimizer	Epochs	Val F1
Light (ours)	HFlip	Adam	10	0.9076
Strong	HFlip+Crop+CJ+RE	AdamW	15	0.8947

TABLE IV
EFFECT OF INFERENCE-TIME STRATEGIES ON VALIDATION F1.

Strategy	Threshold	Val F1	Δ vs. baseline
Baseline	0.50	0.9076	-
TTA only	0.50	0.9053	-0.0023
TTA + threshold search	0.48	0.9066	-0.0010
Threshold search only	0.53	0.9085	+0.0009

for variation. The convergence behavior of our light-augmentation run, training loss reaching 0.019 by epoch 8 while validation F1 continues to improve, suggests the model is learning meaningful generalization rather than memorizing the training set.

We therefore retained the light-augmentation configuration for our final model.

D. Ablation: Inference-Time Strategies

Starting from the best model identified in Section V-B (EfficientNetV2-S, validation F1 = 0.9076 at threshold 0.5, no TTA), we evaluated the effect of two commonly-recommended inference-time enhancements: test-time augmentation (TTA) and threshold tuning. Results are summarized in Table IV.

TTA does not help. Averaging the predictions on the original image and its horizontally-flipped version decreased validation F1 by 0.0023. The reason is straightforward: random horizontal flipping was already part of our training augmentation, so the model had effectively seen flipped inputs during training and learned to produce stable predictions on them. Flipping the image again at inference time amounts to asking the model to vote twice on inputs it can already handle correctly, no new information is added, and the averaging step instead introduces a small amount of jitter.

Threshold tuning is not statistically meaningful. Sweeping the threshold over $[0.30, 0.70]$ identified 0.53 as the validation-optimal value, yielding F1 = 0.9085, an improvement of 0.0009 over the default. However, this difference corresponds to only one image out of 960 having its predicted label flipped. F1 on a validation set of this size carries a natural fluctuation of several thousandths, roughly equivalent to 5-10 images worth of label changes, simply due to which specific images happen to be in the set. An apparent gain from a single image flip is far smaller than this natural fluctuation and cannot be taken as evidence that 0.53 is genuinely better than 0.5.

Final choice. We retained the default threshold of 0.5 for our final submission, both to avoid overfitting to validation-set-specific noise and to keep the inference pipeline simple and reproducible. The final reported validation F1 is 0.9076.

The Phase 3 results partially contradict the Phase 2 ablation findings. Phase 2 found that aggressive augmentation (RandomCrop, ColorJitter, RandomErasing) reduced F1 by 1.3 points. Phase 3 uses the same aggressive pipeline but achieves substantially higher F1. The likely explanation is that Phase 3 compensates through greater model capacity (ConvNeXt-Base has 4× the parameters of EfficientNetV2-S) and the Focal Loss formulation, which is more robust to augmentation noise than standard BCE.

VI. CONCLUSION

We presented a systematic comparison of hybrid feature extraction and end-to-end fine-tuning for detecting AI-generated product images on the Etsy marketplace. Our best model, an EfficientNetV2-S fine-tuned end-to-end with minimal augmentation, achieves a validation F1 of 0.9076, exceeding the strongest Phase 1 hybrid model by 9.4 percentage points. We further evaluated three commonly recommended enhancements, strong data augmentation, test-time augmentation, and threshold tuning, through controlled ablations, and found that none of them produced meaningful gains on this task; the first two in fact reduced performance.

A subsequent Phase 3 investigation extended this work with a three-model ensemble (EfficientNetV2-S+FFT, ConvNeXt-Small, ConvNeXt-Base) trained with Focal Loss, Mixup, and SWA, achieving a validation F1 of 0.9317. Notably, ConvNeXt-Base alone achieved 0.9356, suggesting that for this task, model capacity and architectural choice (large-kernel convolutions) may be more impactful than ensemble diversity. The finding that the three-model ensemble underperformed its strongest constituent is a practically important result: ensemble benefits are not guaranteed when component models differ substantially in quality.

The direction worth pursuing in future work would be CLIP-based universal detectors have recently shown strong generalization across different generative models and may transfer well to this marketplace setting.

ACKNOWLEDGMENT

The authors acknowledge the use of AI assistants (Gemini) for language polishing, LaTeX formatting, and structural suggestions during the writing process. All experimental designs, model training, results, and scientific interpretations in this report are the authors' own work.

REFERENCES

- [1] M. Tan and Q. Le, "Efficientnetv2: Smaller models and faster training," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 10 096–10 106. [Online]. Available: <https://proceedings.mlr.press/v139/tan21a.html>
- [2] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 6105–6114. [Online]. Available: <https://proceedings.mlr.press/v97/tan19a.html>
- [3] M. Kimura, "Understanding test-time augmentation," in *Neural Information Processing*, T. Mantoro, M. Lee, M. A. Ayu, K. W. Wong, and A. N. Hidayanto, Eds. Cham: Springer International Publishing, 2021, pp. 558–569.
- [4] R. Wightman, "Pytorch image models," <https://github.com/rwightman/pytorch-image-models>, 2019.
- [5] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022, pp. 11 976–11 986.
- [6] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2980–2988.
- [7] P. Izmailov, D. Podoprikin, T. Garipov, D. Vetrov, and A. G. Wilson, "Averaging weights leads to wider optima and better generalization," *arXiv preprint arXiv:1803.05407*, 2018.
- [8] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," *arXiv preprint arXiv:1710.09412*, 2017.